Troubleshooting managed feature store

In this article, learn how to troubleshoot common problems you might encounter with the managed feature store in Azure Machine Learning.

Issues found when creating and updating a feature store

You might encounter these issues when you create or update a feature store:

- ARM Throttling Error
- Duplicated Materialization Identity ARM ID Issue
- RBAC Permission Errors
- Older versions of azure-mgmt-authorization package don't work with AzureMLOnBehalfOfCredential

ARM Throttling Error

Symptom

Feature store creation or update fails. The error might look like this:

JSON

```
{
  "error": {
    "code": "TooManyRequests",
    "message": "The request is being throttled as the limit has been reached for oper-
ation type - 'Write'. ..",
    "details": [
      {
        "code": "TooManyRequests",
        "target": "Microsoft.MachineLearningServices/workspaces",
        "message": "..."
      }
    ]
  }
}
```

Solution

Run the feature store create/update operation at a later time. The deployment occurs in multiple steps, so the second attempt might fail because some of the resources already exist. Delete those

resources and resume the job.

Duplicated materialization identity ARM ID issue

Once the feature store is updated to enable materialization for the first time, some later updates on the feature store might result in this error.

Symptom

When the feature store is updated using the SDK/CLI, the update fails with this error message:

Error:

JSON

```
{
  "error":{
    "code": "InvalidRequestContent",
    "message": "The request content contains duplicate JSON property names creating ambiguity in paths 'identity.userAssignedIdentities['/subscriptions/{sub-id}/resourceGroups/{rg}/providers/Microsoft.ManagedIdentity/userAssignedIdentities/{your-uai}']'. Please update the request content to eliminate duplicates and try again."
  }
}
```

Solution

The issue involves the ARM ID of the `materialization_identity` ARM ID format.

From the Azure UI or SDK, the ARM ID of the user-assigned managed identity uses lower case `resourcegroups`. See this example:

- (A): /subscriptions/{sub-id}/**resourcegroups**/{rg}/providers/Microsoft.ManagedIdentity/userAssignedIdentities/{your-uai}

When the feature store uses the user-assigned managed identity as its materialization_identity, its ARM ID is normalized and stored, with `resourceGroups`. See this example:

- (B): /subscriptions/{sub-id}/**resourceGroups**/{rg}/providers/Microsoft.ManagedIdentity/userAssignedIdentities/{your-uai}

In the update request, you might use a user-assigned identity that matches the materialization identity, to update the feature store. While you use the ARM ID in format (A), when you use that managed identity for that purpose, the update fails and it returns the earlier error message.

To fix the issue, replace the string `resourcegroups` with `resourceGroups` in the user-assigned managed identity ARM ID. Then, run the feature store update again.

RBAC permission errors

To create a feature store, the user needs the `Contributor` and `User Access Administrator` roles. A custom role that covers the same set of roles of the actions will work. A super set of those two roles of the actions will also work.

Symptom

If the user doesn't have the required roles, the deployment fails. The error response might look like this:

JSON

```
{
  "error": {
    "code": "AuthorizationFailed",
    "message": "The client '{client_id}' with object id '{object_id}' does not have
authorization to perform action '{action_name}' over scope '{scope}' or the scope is
invalid. If access was recently granted, please refresh your credentials."
  }
}
```

Solution

Grant the `Contributor` and `User Access Administrator` roles to the user on the resource group where the feature store are created. Then, instruct the user to run the deployment again.

For more information, visit the Permissions required for the feature store materialization managed identity role resource.

Older versions of azure-mgmt-authorization package don't work with AzureMLOnBehalfOfCredential

Symptom

In the azureml-examples repository, When you use the `setup_storage_uai` script provided in the *featurestore_sample* folder, the script fails with this error message:

```
AttributeError: 'AzureMLOnBehalfOfCredential' object has no attribute 'signed_session'
```

Solution:

Check the version of the installed `azure-mgmt-authorization` package, and verify that you're using a recent version, at least 3.0.0 or later. An older version - for example 0.61.0 - doesn't work with `AzureMLOnBehalfOfCredential`.

Feature Set Spec Create Errors

- Invalid schema in feature set spec
- Can't find the transformation class
- FileNotFoundError on code folder

Invalid schema in feature set spec

Before you register a feature set into the feature store, define the feature set spec locally, and run `<feature_set_spec>.to_spark_dataframe()` to validate it.

Symptom

When a user runs `<feature_set_spec>.to_spark_dataframe()`, various schema validation failures can occur if the feature set dataframe schema doesn't align with the feature set spec definition.

For example:

- Error message: `azure.ai.ml.exceptions.ValidationException: Schema check errors, timestamp column: timestamp is not in output dataframe`
- Error message: `Exception: Schema check errors, no index column: accountID in output dataframe`
- Error message: `ValidationException: Schema check errors, feature column: transaction_7d_count has data type: ColumnType.long, expected: ColumnType.string`

Solution

Check the schema validation failure error, and update the feature set spec definition accordingly, for both the column names and types. For example:

- update the `source.timestamp_column.name` property to correctly define the timestamp column names
- update the `index_columns` property to correctly define the index columns
- update the `features` property to correctly define the feature column names and types
- if the feature source data is of type *csv*, verify that the CSV files are generated with column headers

Next, run `<feature_set_spec>.to_spark_dataframe()` again to check if the validation passed.

Instead of manually typing in the values, if the SDK defines the feature set spec, the `infer_schema` option is also recommended as the preferred way to autofill the `features`. The `timestamp_column` and `index columns` can't be autofilled.

For more information, visit the Feature Set Spec schema resource.

Can't find the transformation class

Symptom

When a user runs `<feature_set_spec>.to_spark_dataframe()`, it returns this error: `AttributeError: module '<...>' has no attribute '<...>'`

For example:

- `AttributeError: module '7780d27aa8364270b6b61fed2a43b749.transaction_transform' has no attribute 'TransactionFeatureTransformer1'`

Solution

The feature transformation class is expected to have its definition in a Python file under the root of the code folder. The code folder can have other files or sub folders.

Set the value of the `feature_transformation_code.transformation_class` property to `<py file name of the transformation class>.<transformation class name>.`

For example, if the code folder looks like this

```
code/
└── my_transformation_class.py
```

and the my_transformation_class.py file defines the `MyFeatureTransformer` class, set

`feature_transformation_code.transformation_class` to be `my_transformation_class.MyFeatureTransformer`

FileNotFoundError on code folder

Symptom

This error can happen if the feature set spec YAML is manually created, and the SDK doesn't generate the feature set. The command

`runs <feature_set_spec>.to_spark_dataframe()`

returns error

```
FileNotFoundError: [Errno 2] No such file or directory: ....
```

Solution

Check the code folder. It should be a subfolder under the feature set spec folder. In the feature set spec, set `feature_transformation_code.path` as a relative path to the feature set spec folder. For example:

```
feature set spec folder/
├── code/
│   ├── my_transformer.py
│   └── my_orther_folder
└── FeatureSetSpec.yaml
```

In this example, the `feature_transformation_code.path` property in the YAML should be `./code`

> **Note**
>
> When you use the **create_feature_set_spec function** in `azureml-featurestore` to create a FeatureSetSpec python object, it can take any local folder as the `feature_transformation_code.path` value. When the FeatureSetSpec object is dumped to form a feature set spec yaml in a target folder, the code path is copied into the target folder, and the `feature_transformation_code.path` property is updated in the spec yaml.

Feature set CRUD Errors

Feature set GET fails due to invalid FeatureStoreEntity

Symptom

When you use the feature store CRUD client to GET a feature set - for example, `fs_client.feature_sets.get(name, version)"` - you might get this error:

Python

```
Traceback (most recent call last):

  File "/home/trusted-service-user/cluster-env/env/lib/python3.8/site-
packages/azure/ai/ml/operations/_feature_store_entity_operations.py", line 116, in get

    return FeatureStoreEntity._from_rest_object(feature_store_entity_version_resource)
```

```
  File "/home/trusted-service-user/cluster-env/env/lib/python3.8/site-
packages/azure/ai/ml/entities/_feature_store_entity/feature_store_entity.py", line 93,
in _from_rest_object

    featurestoreEntity = FeatureStoreEntity(

  File "/home/trusted-service-user/cluster-env/env/lib/python3.8/site-
packages/azure/ai/ml/_utils/_experimental.py", line 42, in wrapped

    return func(*args, **kwargs)

  File "/home/trusted-service-user/cluster-env/env/lib/python3.8/site-
packages/azure/ai/ml/entities/_feature_store_entity/feature_store_entity.py", line 67,
in __init__

    raise ValidationException(

azure.ai.ml.exceptions.ValidationException: Stage must be Development, Production, or
Archived, found None
```

This error can also happen in the FeatureStore materialization job, where the job fails with the same error trace back.

Solution

Start a notebook session with the new version of SDKS

- If it uses azure-ai-ml, update to `azure-ai-ml==1.8.0`.
- If it uses the feature store dataplane SDK, update it to `azureml-featurestore== 0.1.0b2`.

In the notebook session, update the feature store entity to set its `stage` property, as shown in this example:

Python

```python
from azure.ai.ml.entities import DataColumn, DataColumnType

account_entity_config = FeatureStoreEntity(

    name="account",

    version="1",

    index_columns=[DataColumn(name="accountID", type=DataColumnType.STRING)],

    stage="Development",

    description="This entity represents user account index key accountID.",

    tags={"data_typ": "nonPII"},

)
```

```
poller =
fs_client.feature_store_entities.begin_create_or_update(account_entity_config)

print(poller.result())
```

When you define the FeatureStoreEntity, set the properties to match the properties used when it was created. The only difference is to add the `stage` property.

Once the `begin_create_or_update()` call returns successfully, the next `feature_sets.get()` call and the next materialization job should succeed.

Feature Retrieval job and query errors

- Feature Retrieval Specification Resolution Errors
- File *feature_retrieval_spec.yaml* not found when using a model as input to the feature retrieval job
- Observation Data isn't Joined with any feature values
- User or Managed Identity doesn't have proper RBAC permission on the feature store
- User or Managed Identity doesn't have proper RBAC permission to Read from the Source Storage or Offline store
- Training job fails to read data generated by the built-in Feature Retrieval Component
- generate_feature_retrieval_spec() fails due to use of local feature set specification
- The get_offline_features() query takes a long time
- Online feature lookup initialization fails due to missing storage scope

When a feature retrieval job fails, check the error details. Go to the **run detail page**, select the **Outputs + logs** tab, and examine the *logs/azureml/driver/stdout* file.

If user runs the `get_offline_feature()` query in the notebook, cell outputs directly show the error.

Feature retrieval specification resolution errors

Symptom

The feature retrieval query/job shows these errors:

- Invalid feature

JSON

```
code: "UserError"
mesasge: "Feature '<some name>' not found in this featureset."
```

- Invalid feature store URI:

JSON

```
message: "the Resource 'Microsoft.MachineLearningServices/workspaces/<name>' under re-
source group '<>>resource group name>'->' was not found. For more details please go to
https://aka.ms/ARMResourceNotFoundFix",
code: "ResourceNotFound"
```

- Invalid feature set:

JSON

```
code: "UserError"
message: "Featureset with name: <name >and version: <version> not found."
```

Solution

Check the content in the `feature_retrieval_spec.yaml` that the job uses. Verify that all the feature store names

- URI
- feature set name/version
- feature

are valid and that they exist in the feature store.

To select features from a feature store and generate the feature retrieval spec YAML file, use of the utility function is recommended.

This code snippet uses the `generate_feature_retrieval_spec` utility function.

Python

```
from azureml.featurestore import FeatureStoreClient
from azure.ai.ml.identity import AzureMLOnBehalfOfCredential

featurestore = FeatureStoreClient(
credential = AzureMLOnBehalfOfCredential(),
subscription_id = featurestore_subscription_id,
resource_group_name = featurestore_resource_group_name,
name = featurestore_name
)

transactions_featureset = featurestore.feature_sets.get(name="transactions", version =
"1")

features = [
    transactions_featureset.get_feature('transaction_amount_7d_sum'),
```

```
    transactions_featureset.get_feature('transaction_amount_3d_sum')
]

feature_retrieval_spec_folder = "./project/fraud_model/feature_retrieval_spec"
featurestore.generate_feature_retrieval_spec(feature_retrieval_spec_folder, features)
```

File *feature_retrieval_spec.yaml* not found when using a model as input to the feature retrieval job

Symptom

When you use a registered model as a feature retrieval job input, the job fails with this error:

Python

```
ValueError: Failed with visit error: Failed with execution error: error in streaming
from input data sources
    VisitError(ExecutionError(StreamError(NotFound)))
=> Failed with execution error: error in streaming from input data sources
    ExecutionError(StreamError(NotFound)); Not able to find path:
azureml://subscriptions/{sub_id}/resourcegroups/{rg}/workspaces/{ws}/datastores/worksp
aceblobstore/paths/LocalUpload/{guid}/feature_retrieval_spec.yaml
```

Solution:

When you provide a model as input to the feature retrieval step, the model expects to find the retrieval spec YAML file under the model artifact folder. The job fails if that file is missing.

To fix the issue, package the `feature_retrieval_spec.yaml` file in the root folder of the model artifact folder before you register the model.

Observation Data isn't joined with any feature values

Symptom

After users run the feature retrieval query/job, the output data gets no feature values. For example, a user runs the feature retrieval job to retrieve the `transaction_amount_3d_avg` and `transaction_amount_7d_avg` features, with these results:

Expand table

transactionID	accountID	timestamp	is_fraud	transaction_amount_3d_avg	transaction_amount_7
83870774-7A98-43B...	A1055520444618950	2023-02-28 04:34:27	0	null	null

transactionID	accountID	timestamp	is_fraud	transaction_amount_3d_avg	transaction_amount_7
25144265-F68B-4FD...	A1055520444618950	2023-02-28 10:44:30	0	null	null
8899ED8C-B295-43F...	A1055520444812380	2023-03-06 00:36:30	0	null	null

Solution

Feature retrieval does a point-in-time join query. If the join result shows empty, try these potential solutions:

- Extend the `temporal_join_lookback` range in the feature set spec definition, or temporarily remove it. This allows the point-in-time join to look back further (or infinitely) into the past, before the observation event time stamp, to find the feature values.
- If `source.source_delay` is also set in the feature set spec definition, make sure that `temporal_join_lookback > source.source_delay`.

If none of these solutions work, get the feature set from feature store, and run `<feature_set>.to_spark_dataframe()` to manually inspect the feature index columns and timestamps. The failure could happen because:

- the index values in the observation data don't exist in the feature set dataframe
- no feature value, with a timestamp value before the observation timestamp, exists.

In these cases, if the feature enabled offline materialization, you might need to backfill more feature data.

User or managed identity doesn't have proper RBAC permission on the feature store

Symptom:

The feature retrieval job/query fails with this error message in the *logs/azureml/driver/stdout* file:

Python

```
Traceback (most recent call last):
  File "/home/trusted-service-user/cluster-env/env/lib/python3.8/site-
packages/azure/ai/ml/_restclient/v2022_12_01_preview/operations/_workspaces_operations
.py", line 633, in get
    raise HttpResponseError(response=response, model=error,
error_format=ARMErrorFormat)
azure.core.exceptions.HttpResponseError: (AuthorizationFailed) The client 'XXXX' with
object id 'XXXX' does not have authorization to perform action
```

```
'Microsoft.MachineLearningServices/workspaces/read' over scope
'/subscriptions/XXXX/resourceGroups/XXXX/providers/Microsoft.MachineLearningServices/w
orkspaces/XXXX' or the scope is invalid. If access was recently granted, please re-
fresh your credentials.
Code: AuthorizationFailed
```

Solution:

1. If the feature retrieval job uses a managed identity, assign the `AzureML Data Scientist` role on the feature store to the identity.

2. If the problem happens when

- the user runs code in an Azure Machine Learning Spark notebook
- that notebook uses the user's own identity to access the Azure Machine Learning service

assign the `AzureML Data Scientist` role on the feature store to the user's Microsoft Entra identity.

`Azure Machine Learning Data Scientist` is a recommended role. User can create their own custom role with the following actions

- Microsoft.MachineLearningServices/workspaces/datastores/listsecrets/action
- Microsoft.MachineLearningServices/workspaces/featuresets/read
- Microsoft.MachineLearningServices/workspaces/read

For more information about RBAC setup, visit the Manage access to managed feature store resource.

User or Managed Identity doesn't have proper RBAC permission to Read from the Source Storage or Offline store

Symptom

The feature retrieval job/query fails with the following error message in the *logs/azureml/driver/stdout* file:

Python

```
An error occurred while calling o1025.parquet.
: java.nio.file.AccessDeniedException: Operation failed: "This request is not autho-
rized to perform this operation using this permission.", 403, GET,
https://{storage}.dfs.core.windows.net/test?
upn=false&resource=filesystem&maxResults=5000&directory=datasources&timeout=90&recursi
ve=false, AuthorizationPermissionMismatch, "This request is not authorized to perform
this operation using this permission. RequestId:63013315-e01f-005e-577b-7c63b8000000
Time:2023-05-01T22:20:51.1064935Z"
    at
```

```
org.apache.hadoop.fs.azurebfs.AzureBlobFileSystem.checkException(AzureBlobFileSystem.j
ava:1203)
    at
org.apache.hadoop.fs.azurebfs.AzureBlobFileSystem.listStatus(AzureBlobFileSystem.java:
408)
    at org.apache.hadoop.fs.Globber.listStatus(Globber.java:128)
    at org.apache.hadoop.fs.Globber.doGlob(Globber.java:291)
    at org.apache.hadoop.fs.Globber.glob(Globber.java:202)
    at org.apache.hadoop.fs.FileSystem.globStatus(FileSystem.java:2124)
```

Solution:

- If the feature retrieval job uses a managed identity, assign the `Storage Blob Data Reader` role on both the source storage and offline store storage to the identity.
- This error happens when the notebook uses the user's identity to access the Azure Machine Learning service to run the query. To resolve the error, assign the `Storage Blob Data Reader` role to the user's identity on the source storage and offline store storage account.

`Storage Blob Data Reader` is the minimum recommended access requirement. Users can also assign roles - for example, `Storage Blob Data Contributor` or `Storage Blob Data Owner` - with more privileges.

Training job fails to read data generated by the built-in Feature Retrieval Component

Symptom

A training job fails with the error message that the training data doesn't exist, the format is incorrect, or there's a parser error:

JSON

```
FileNotFoundError: [Errno 2] No such file or directory
```

- format isn't correct.

JSON

```
ParserError:
```

Solution

The built-in feature retrieval component has one output, `output_data`. The output data is a uri_folder data asset. It always has this folder structure:

```
<training data folder>/
├── data/
│  ├── xxxxx.parquet
│  └── xxxxx.parquet
└── feature_retrieval_spec.yaml
```

The output data is always in parquet format. Update the training script to read from the "data" sub folder, and read the data as parquet.

`generate_feature_retrieval_spec()` fails due to use of local feature set specification

Symptom:

This python code generates a feature retrieval spec on a given list of features:

Python

```
featurestore.generate_feature_retrieval_spec(feature_retrieval_spec_folder, features)
```

If the features list contains features defined by a local feature set specification, the `generate_feature_retrieval_spec()` fails with this error message:

```
AttributeError: 'FeatureSetSpec' object has no attribute 'id'
```

Solution:

A feature retrieval spec can only be generated using feature sets registered in Feature Store. To fix the problem:

- Register the local feature set specification as a feature set in the feature store
- Get the registered feature set
- Create feature lists again using only features from registered feature sets
- Generate the feature retrieval spec using the new features list

The `get_offline_features()` query takes a long time

Symptom:

Running `get_offline_features` to generate training data, using a few features from feature store, takes too long to finish.

Solutions:

Check these configurations:

- Verify that each feature set used in the query has `temporal_join_lookback` set in the feature set specification. Set its value to a smaller value.
- If the size and timestamp window on the observation dataframe are large, configure the notebook session (or the job) to increase the size (memory and core) of the driver and executor. Additionally, increase the number of executors.

Online feature lookup initialization fails due to missing storage scope

Symptom

When you call `init_online_lookup()` to initialize lookup of online features, the operation fails with this error:

Python

```
KeyError: 'https://storage.azure.com/.default'
```

Solution

Upgrade the feature store SDK to version 1.2.1 or later, which contains the necessary storage account permissions and scopes. Earlier versions of the SDK lack the appropriate storage scopes required for online feature retrieval.

To upgrade the SDK:

Python

```
pip install azureml-featurestore>=1.2.1
```

After upgrading, initialize online lookup using:

Python

```
init_online_lookup(features, AzureMLOnBehalfOfCredential())
```

Feature Materialization Job Errors

- Invalid Offline Store Configuration
- Materialization Identity doesn't have the proper RBAC permission on the feature store
- Materialization Identity doesn't have proper RBAC permission to read from the Storage

- Materialization identity doesn't have RBAC permission to write data to the offline store
- Streaming job execution results to a notebook results in failure
- Invalid Spark configuration

When the feature materialization job fails, follow these steps to check the job failure details:

1. Navigate to the feature store page: https://ml.azure.com/featureStore/{your-feature-store-name} .
2. Go to the `feature set` tab, select the relevant feature set, and navigate to the **Feature set detail page**.
3. From feature set detail page, select the `Materialization jobs` tab, then select the failed job to open it in the job details view.
4. On the job detail view, under the `Properties` card, review the job status and error message.
5. You can also go to the `Outputs + logs` tab, then find the `stdout` file from the *`logs\azureml\driver\stdout`* file.

After you apply a fix, you can manually trigger a backfill materialization job to verify that the fix works.

Invalid Offline Store Configuration

Symptom

The materialization job fails with this error message in the *`logs/azureml/driver/stdout`* file:

JSON

```
Caused by: Status code: -1 error code: null error message:
InvalidAbfsRestOperationExceptionjava.net.UnknownHostException:
adlgen23.dfs.core.windows.net
```

JSON

```
java.util.concurrent.ExecutionException: Operation failed: "The specified resource
name contains invalid characters.", 400, HEAD,
https://{storage}.dfs.core.windows.net/{container-name}/{fs-
id}/transactions/1/_delta_log?upn=false&action=getStatus&timeout=90
```

Solution

Use the SDK to check the offline storage target defined in the feature store:

Python

```
from azure.ai.ml import MLClient
from azure.ai.ml.identity import AzureMLOnBehalfOfCredential

fs_client = MLClient(AzureMLOnBehalfOfCredential(), featurestore_subscription_id,
featurestore_resource_group_name, featurestore_name)

featurestore = fs_client.feature_stores.get(name=featurestore_name)
featurestore.offline_store.target
```

You can also check the offline storage target on the feature store UI overview page. Verify that both the storage and container exist, and that the target has this format:

*/subscriptions/{sub-id}/resourceGroups/{rg}/providers/Microsoft.Storage/storageAccounts/{**storage**}/blobServices/default/containers/{**container-name**}*

Materialization Identity doesn't have proper RBAC permission on the feature store

Symptom:

The materialization job fails with this error message in the *logs/azureml/driver/stdout* file:

Python

```
Traceback (most recent call last):
  File "/home/trusted-service-user/cluster-env/env/lib/python3.8/site-
packages/azure/ai/ml/_restclient/v2022_12_01_preview/operations/_workspaces_operations
.py", line 633, in get
    raise HttpResponseError(response=response, model=error,
error_format=ARMErrorFormat)
azure.core.exceptions.HttpResponseError: (AuthorizationFailed) The client 'XXXX' with
object id 'XXXX' does not have authorization to perform action
'Microsoft.MachineLearningServices/workspaces/read' over scope
'/subscriptions/XXXX/resourceGroups/XXXX/providers/Microsoft.MachineLearningServices/w
orkspaces/XXXX' or the scope is invalid. If access was recently granted, please re-
fresh your credentials.
Code: AuthorizationFailed
```

Solution:

Assign the `Azure Machine Learning Data Scientist` role on the feature store to the materialization identity (a user assigned managed identity) of the feature store.

`Azure Machine Learning Data Scientist` is a recommended role. You can create your own custom role with these actions:

- Microsoft.MachineLearningServices/workspaces/datastores/listsecrets/action
- Microsoft.MachineLearningServices/workspaces/featuresets/read
- Microsoft.MachineLearningServices/workspaces/read

For more information, visit the Permissions required for the feature store materialization managed identity role resource.

Materialization identity doesn't have proper RBAC permission to read from the storage

Symptom

The materialization job fails with this error message in the *logs/azureml/driver/stdout* file:

Python

```
An error occurred while calling o1025.parquet.
: java.nio.file.AccessDeniedException: Operation failed: "This request is not autho-
rized to perform this operation using this permission.", 403, GET,
https://{storage}.dfs.core.windows.net/test?
upn=false&resource=filesystem&maxResults=5000&directory=datasources&timeout=90&recursi
ve=false, AuthorizationPermissionMismatch, "This request is not authorized to perform
this operation using this permission. RequestId:63013315-e01f-005e-577b-7c63b8000000
Time:2023-05-01T22:20:51.1064935Z"
    at
org.apache.hadoop.fs.azurebfs.AzureBlobFileSystem.checkException(AzureBlobFileSystem.j
ava:1203)
    at
org.apache.hadoop.fs.azurebfs.AzureBlobFileSystem.listStatus(AzureBlobFileSystem.java:
408)
    at org.apache.hadoop.fs.Globber.listStatus(Globber.java:128)
    at org.apache.hadoop.fs.Globber.doGlob(Globber.java:291)
    at org.apache.hadoop.fs.Globber.glob(Globber.java:202)
    at org.apache.hadoop.fs.FileSystem.globStatus(FileSystem.java:2124)
```

Solution:

Assign the `Storage Blob Data Reader` role, on the source storage, to the materialization identity (a user-assigned managed identity) of the feature store.

`Storage Blob Data Reader` is the minimum recommended access requirement. You can also assign roles with more privileges - for example, `Storage Blob Data Contributor` or `Storage Blob Data Owner`.

For more information about RBAC configuration, visit the Permissions required for the feature store materialization managed identity role resource.

Materialization identity doesn't have proper RBAC permission to write data to the offline store

Symptom

The materialization job fails with this error message in the *logs/azureml/driver/stdout* file:

YAML

```
An error occurred while calling o1162.load.
: java.util.concurrent.ExecutionException: java.nio.file.AccessDeniedException:
Operation failed: "This request is not authorized to perform this operation using this
permission.", 403, HEAD,
https://featuresotrestorage1.dfs.core.windows.net/offlinestore/fs_xxxxxx-xxxx-xxxx-
xxxx-xxxxxxxxxxxx_fsname/transactions/1/_delta_log?
upn=false&action=getStatus&timeout=90
    at
com.google.common.util.concurrent.AbstractFuture$Sync.getValue(AbstractFuture.java:306
)
    at
com.google.common.util.concurrent.AbstractFuture$Sync.get(AbstractFuture.java:293)
    at com.google.common.util.concurrent.AbstractFuture.get(AbstractFuture.java:116)
    at
com.google.common.util.concurrent.Uninterruptibles.getUninterruptibly(Uninterruptibles
.java:135)
    at
com.google.common.cache.LocalCache$Segment.getAndRecordStats(LocalCache.java:2410)
    at com.google.common.cache.LocalCache$Segment.loadSync(LocalCache.java:2380)
    at com.google.common.cache.LocalCache$S
```

Solution

Assign the `Storage Blob Data Reader` role on the source storage to the materialization identity (a user-assigned managed identity) of the feature store.

`Storage Blob Data Contributor` is the minimum recommended access requirement. You can also assign roles with more privileges - for example, `Storage Blob Data Owner`.

For more information about RBAC configuration, visit the Permissions required for the feature store materialization managed identity role resource.

Streaming job output to a notebook results in failure

Symptom:

When using the feature store CRUD client to stream materialization job results to a notebook, using `fs_client.jobs.stream("<job_id>")`, the SDK call fails with an error

```
HttpResponseError: (UserError) A job was found, but it is not supported in this API
version and cannot be accessed.

Code: UserError

Message: A job was found, but it is not supported in this API version and cannot be
accessed.
```

Solution:

When the materialization job is created (for example, by a backfill call), it might take a few seconds for the job to properly initialize. Run the `jobs.stream()` command again a few seconds later. This should resolve the issue.

Invalid Spark configuration

Symptom:

A materialization job fails with this error message:

Python

```
Synapse job submission failed due to invalid spark configuration request

{

"Message":"[..] Either the cores or memory of the driver, executors exceeded the
SparkPool Node Size.\nRequested Driver Cores:[4]\nRequested Driver Memory:
[36g]\nRequested Executor Cores:[4]\nRequested Executor Memory:[36g]\nSpark Pool Node
Size:[small]\nSpark Pool Node Memory:[28]\nSpark Pool Node Cores:[4]"

}
```

Solution:

Update the `materialization_settings.spark_configuration{}` of the feature set. Make sure that these parameters use both memory size amounts, and a total number of core values, that are both less than what the instance type, as defined by `materialization_settings.resource`, provides:

`spark.driver.cores` `spark.driver.memory` `spark.executor.cores` `spark.executor.memory`

For example, for instance type *standard_e8s_v3*, this Spark configuration is one of the valid options:

Python

```
transactions_fset_config.materialization_settings = MaterializationSettings(
    offline_enabled=True,
    resource = MaterializationComputeResource(instance_type="standard_e8s_v3"),
    spark_configuration = {
        "spark.driver.cores": 4,
        "spark.driver.memory": "36g",
        "spark.executor.cores": 4,
        "spark.executor.memory": "36g",
        "spark.executor.instances": 2
    },
    schedule = None,
)
fs_poller = fs_client.feature_sets.begin_create_or_update(transactions_fset_config)
```

Next steps

- What is managed feature store?
- Understanding top-level entities in managed feature store

ⓘ **Note:** The author created this article with assistance from AI. Learn more

Last updated on 11/13/2025